

# Proving PyTorch optimizations are correct — automatically

Formal equivalence checking of two PyTorch programs with ESBMC

Lucas Cordeiro

University of Manchester

Proof-of-concept · status update

## The problem, in one sentence

We constantly **rewrite** deep-learning code to make it run faster.

But how do we *know* the faster version computes the **same thing** as the original?

- A rewrite that is *almost* identical can silently change the results.
- Silently wrong math → wrong predictions, bugs that are very hard to find.

We want a way to be **sure**, not hopeful — to **check an optimised version before it ships to production.**

## Our example: the “QKV” step inside every Transformer

Attention models — the **Transformer** design behind modern AI (e.g. ChatGPT) — build three things, **Q**, **K**, **V** (*intermediate matrices that decide what the model “pays attention” to*), from the input  $X$  using three weight matrices.

**The clear way** (unfused) — three separate matrix multiplications:

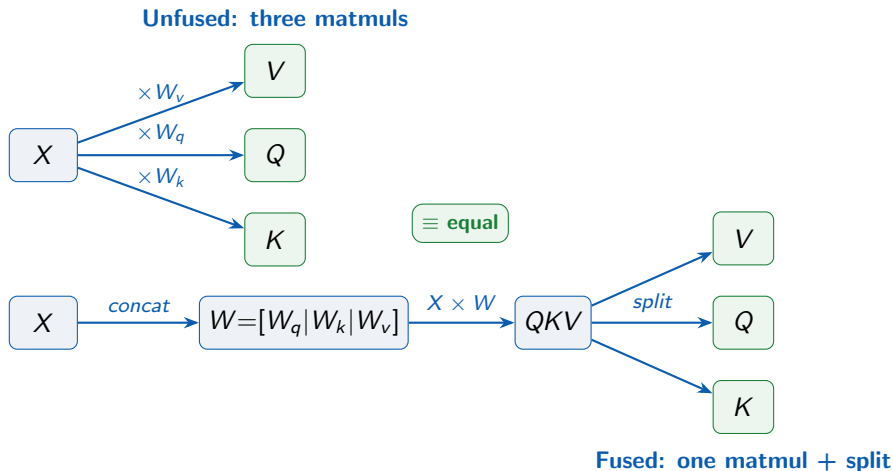
$$Q = X W_q \quad K = X W_k \quad V = X W_v$$

**The fast way** (fused) — glue the weights together, do **one** big multiply, then split:

$$W = [W_q \mid W_k \mid W_v] \quad QKV = X W \quad \rightarrow \text{split into } Q, K, V$$

These **should** give identical results. The question: do they — *always*?

## The two programs, side by side



ESBMC proves these two produce **the same result for every input we model**.  
So: a **faster implementation can be proven to behave exactly like the original**.

We run the program on a **handful of example inputs** and compare.

*Like tasting a few spoonfuls of soup and hoping the whole pot is fine.*

- Testing only covers the inputs you happened to try.
- A bug hiding in some other input is simply **missed**.
- **Passing tests  $\neq$  correct.**

**Testing gives confidence. Proof gives guarantees.**

## What we actually want: a proof, not a sample

Instead of trying a few inputs, check **every input we model** — **all at once** — mathematically.

*“Model” here = a defined tensor size and a bounded input range — not arbitrarily large programs.*

Two possible outcomes:

- ✓ **PROVEN equal** — guaranteed identical for every input in the model.
- ✗ **Counterexample** — one specific input where they differ, handed straight to you.

Either way, you learn something **certain**.

**ESBMC** is an automated *model checker*.

- 1 It **reads the program** and turns the question “*are these two always equal?*” into one giant logic puzzle.
- 2 An automated **solver** (*a program that checks mathematical statements automatically*) then either finds an input that breaks it, or proves that **no such input exists** (within the model).

**Fully automated** once the program is encoded — you write *no* proofs by hand.

The same kind of checking is **widely used in chip design and safety-critical software** (aerospace, automotive, medical).

# The trick that turns a test into a proof

| Ordinary test                                          | Our verification                                                                               |
|--------------------------------------------------------|------------------------------------------------------------------------------------------------|
| one <b>random</b> tensor <code>torch.randn(...)</code> | a <b>symbolic</b> tensor — one value standing for <i>all possible inputs at once</i> (bounded) |
| checks that one case                                   | checks the <b>whole modelled range at once</b>                                                 |
| “looks fine”                                           | a <b>guarantee</b> — <b>within the model</b>                                                   |

We assert `unfused == fused`, feed it *any* input in range, and let ESBMC settle it.

Inputs are bounded to a finite range to exclude “not-a-number”. (→ Assumptions, backup.)

## Result: the example is PROVEN correct

- ✓ Proven equal for **all inputs we model**.
- ✓ Proven **both senses of “equal”**: **bit-for-bit exact** (zero tolerance — literally identical, IEEE-754) *and* within `torch.allclose`’s **real tolerance** (`rtol=1e-5`, `atol=1e-8`).
- ✓ Proven in **two encodings**: plain-math (the matmul written out as explicit loops) *and* the real `torch.mm + torch.allclose`.
- ✓ When we **deliberately break it** (swap two columns), ESBMC **catches it** and shows the exact failing input.

What “modelled” means (fixed sizes, bounded inputs) → **Assumptions & Scope**, backup.

## What the suite covers (and what it doesn't — yet)

**8 / 8** checks behave as expected — small *on purpose*: it is breadth, not scale.

| Dimension           | Coverage                                                                                                                                                                                                                                                                |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Problem classes     | QKV projection · bias-fused linear ( $XW + b$ )                                                                                                                                                                                                                         |
| Encodings           | plain-math <b>and</b> torch-native ( <code>torch.mm</code> / <code>allclose</code> )                                                                                                                                                                                    |
| Strength of “equal” | <b>bit-for-bit exact</b> (zero tol.) <b>and tolerance</b> ( <code>allclose</code> defaults)                                                                                                                                                                             |
| Check type          | a <b>proof</b> (clean $\Rightarrow$  ) <b>and</b> a <b>refutation</b> (bug $\Rightarrow$  + c. ex.) |

Every case is double-checked: we prove the correct version *and* confirm a broken one is caught. **Early-stage PoC** — small fixed sizes for now (see Performance, backup).

- **All modelled inputs**, not a few → no hidden corner cases within the model.
- **Exact** → catches tiny numerical drift that testing would wave through.
- **Automated** → no hand-written proofs.
- **Actionable failures** → it gives you the precise input that breaks things.

Concretely: **detect incorrect optimizations early** and **validate fusion / compiler transformations** *before* they ship.

**In short: we taught ESBMC to understand PyTorch-style code** — 4 contributions merged upstream:

- A torch **operational model** — `mm`, `matmul`, `cat`, `split`, `allclose`.
- Fixes so **matrix math over nested lists works** — values/types survive function returns and copies.
- A **floating-point element-type fix** for matrices built by comprehension (we found, root-caused, and fixed it).

Result: PyTorch-style code is now verifiable in ESBMC **at all** — reusable by anyone.

- ✓ Motivating **QKV** example **fully supported** (two encodings, exact proof).
- ✓ Extended to a **second** class: bias-fused linear ( $XW + b$ ).
- → Today this runs on **small examples**; scaling to large, real-world models is **ongoing work** (see Performance, backup).

## Takeaway

**You can automatically check that a faster version of your code gives exactly the same result — before it ships and causes bugs.**

*Testing samples. This proves — and when it can't, it hands you the bug.*

For all inputs within a defined size and range · today on small examples · scaling is ongoing.

## **Backup slides**

Assumptions · Under the hood · Performance

### What we prove

- Equivalence of the two programs *as written*, for **all inputs in the modelled range**.
- Floating-point modelled **exactly** (IEEE-754, bit-precise) — not real arithmetic, not a hand-waved tolerance.

### What we model / bound

- **Fixed tensor sizes** (bounded model checking; concrete, e.g.  $S=1, D=2, H=1$ ).
- **Inputs bounded** to a finite range, which excludes NaN/Inf.

**What we do not claim:** arbitrary/symbolic sizes · performance equivalence · behaviour outside the bounds · backend-specific optimisations (FMA, parallel reductions, cuBLAS) — we model a sequential reference.

- 1 ESBMC **unrolls** the program (fixed sizes  $\Rightarrow$  finite) and turns each assert into a **verification condition**.
- 2 Tensors are modelled as lists of **IEEE-754 floats**; `torch.mm` / `torch.allclose` have operational models.
- 3 The condition is essentially: *for all bounded inputs, `fused[i][j] == unfused[i][j]`*.
- 4 An **SMT solver** (Bitwuzla / Z3) decides it. (*It searches for a breaking input: none exists — UNSAT  $\Rightarrow$  proved; one found — SAT  $\Rightarrow$  that input is the counterexample.*)

```
X, Wq, Wk, Wv = symbolic, bounded      # any numbers in range
assert torch.allclose(unfused, fused)    # -> one SMT formula over FP
```

| Check                     | Time (Bitwuzla, macOS arm64) |
|---------------------------|------------------------------|
| scalar QKV proof          | ~11 s                        |
| torch-native QKV proof    | ~110 s                       |
| refuting a broken variant | 31–292 s                     |

- Cost grows quickly with tensor size (more multiply-adds → a bigger FP formula).
- `torch.cat` / `torch.split` are modelled but **unwinding-heavy** (a  $1 \times 1 \times 3$  `cat+split`  $\approx 2.5$  min) → we fuse/split by manual indexing for now.
- **Scaling to realistic sizes is the main open bottleneck** (tracked upstream).